ONBOARDING · AGENTIC SYSTEMS ENGINEERING

ASE Team Handbook

ASE takes a written specification and produces an engineered, analysed and optimised agentic system. This handbook explains why it exists, how it is built, where it is going, and the rules everyone on the team works by.

Sources: `CLAUDE.md`, `docs/ROADMAP.md`, `models/*.sysml`, `methodology/*.sysml`, `docs/ase-agent-factory.pdf` (business briefing, October 2026). State as of 28 October 2026, increment I0 delivered.

What ASE is

ASE is an **agent factory**. A client's requirements go in as a specification. ASE consolidates them, picks a proven design from a pattern library, models the design in SysML v2, proves its properties with automated checks, gets a human decision, generates the code, evaluates it, optimises it and, from increment I4, deploys it.

> "The goal is to produce agents the way a plant produces parts: from a library of proven designs, through a fixed routing, with inspection at every station." (business briefing)

Three assets serve that aim, and when priorities conflict, ASE wins:

PRIMARY

ASE Agentic System

The factory itself: models, analysis, code generation, the workbench app, the MCP server.

FEEDS ASE

Pattern knowledge base

The SysML v2 pattern library, the methodology and catalogue, the knowledge graph, and the skill library. ASE selects designs from it; ASE runs feed gaps back into it.

USES ASE

ASE School

An advanced course in agentic architecture engineering. Every module ships a real production asset; "the coursework *is* the factory build".

Business philosophy

The business case rests on one observation: most companies need the same few kinds of agent, and they need them built, checked and maintained reliably rather than invented each time.

The product is five agent types, not industries

`Document Q&A` `Document checker` `Triage & router` `Exception handler` `Report drafter`

Each type is a pattern family in the library. A new client is a new specification for an existing type, not a new project from scratch.

The economic lever is fleet maintenance

"A fix to a template, a security patch or a model upgrade is made once. ASE then regenerates every affected agent, re-runs its tests and redeploys only those that pass." That only works because generated code is deterministic and every agent has its own evals.

Who it is for

- **Mid-sized companies**, including those on smaller ERPs (Business Central, Odoo, Sage, Epicor), as well as large ones.
- **A partner channel**: ERP resellers and managed IT providers who need "a repeatable, tested product they can resell without an AI team".

Commitments to clients

Vendor-neutral

"The LLM provider is configuration." Generated code uses open frameworks (LangGraph, DSPy), so the client owns it.

Governance by design

"The audit trail is a by-product of production, not extra work." Every decision is a recorded graph node.

Nothing ships untested

An agent is deployed only after it passes evals tied to the client's own requirements.

How the factory is measured

METRIC	MEANING
Throughput	Agents delivered per period
First-pass yield	Share of designs that pass all checks and evals without rework
Lead time	Spec in to deployed agent; target "under one day" for a standard type
Cost per agent	Machine and people cost to produce one agent
Rework rate	Share of designs sent back from evaluation (activities A7, A8)

Commercial sequence

WHEN	OFFER
Now	Design sprint (2 to 3 weeks, fixed price), architecture review, training
Next	Agent packs, partner channel, AI assurance and evidence packs (EU AI Act, ISO/IEC 42001)
Later	The factory as a platform

Engineering principles

These rules are not style preferences. Tests and review enforce them, and a change that breaks one is rejected however good it is otherwise.

Six non-negotiables

1

Controls are read-only

The pattern library (`models/rag-patterns.sysml`), `methodology/` and the skill library are obeyed, never edited by an ASE run. Gaps become an `@Extension` on the design, with a reason. A passing spec with a dirty `models/` diff is a failing spec. Promotion into the library is a human commit.

2

ASE is modelled in ASE

`models/ase-system.sysml` describes ASE's own agents, flows and budgets in the same vocabulary it uses for clients. The orchestrator is generated from it, and selftest proves they match.

3

Every decision has a record

Pattern choice, waiver, deferral, assumption, optimiser acceptance: each is a graph node with owner, date, reason and the requirement it affects. "ASE drafts options and revisions; it never ratifies one."

4

Generated artifacts are never edited

`generated/` and `ASE School/notebooks/` are build outputs. Change the generator, then regenerate.

5

Skills live in one place

Only in `AI Skills/skills/` . Architecture skills map to the methodology and are read through the `ase` MCP tools.

6

Provider-agnostic

The model is chosen by `SYSMLAGENTS_MODEL` (a `provider/model` string) through `dspy.LM` or a LangChain factory. No provider SDK in pipeline, agent or codegen code. Offline paths work with no key.

The production contract

ASE's output is a deployed agent, not a source file. Three rules make that repeatable:

1. **Code is deterministic.** Same design, same library version, same generator version gives a byte-identical program and deployment package.

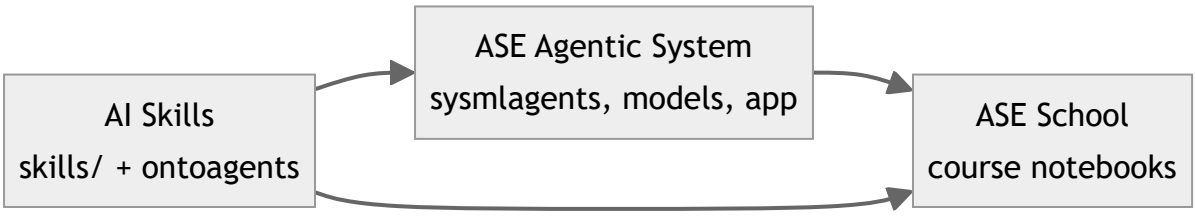
2. **What a model produces is governed, not deterministic.** Eval datasets and optimised prompts are versioned artifacts with seed, model, input hashes and approver. A regenerated artifact is a new version, never an overwrite.
3. **Nothing is deployed that has not passed its evals.** The deploy gate reads the requirement thresholds and the held-out verdict.

Where the model decides, and where it doesn't

Deterministic where the model determines the answer: code generation, analysis, traceability, evidence packs, deployment, and deciding *what* is missing from a spec. Model-driven where it does not: wording elicitation questions, writing rationale, composing new designs, synthesising datasets, writing instructions.

Repository map

Three projects with one-way dependencies. There is no install step and no packaging; `sysmlagents/_paths.py` puts `AI Skills` on the import path.

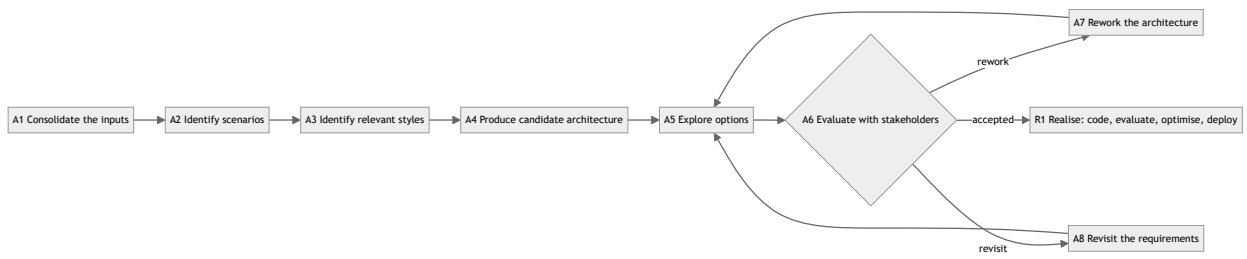


PATH	WHAT LIVES THERE
`AI Skills/skills/`	32 skills: architecture process (`ase-arch-*`), 9 viewpoints, 13 perspectives, the 55-pattern catalogue, the SysML gate (`description-to-sysmlv2`)
`AI Skills/ontoagents/`	Shared substrate: `skill_port` (progressive disclosure of skills), `lc_tools` (skills as LangChain tools), `observability` (traces for LangChain and DSPy)
`ASE Agentic System/models/`	Pattern library, ASE self-model, evolution model, tripwire model
`ASE Agentic System/methodology/`	IDEF0 process, viewpoint and perspective catalogue, profile

PATH	WHAT LIVES THERE
`ASE Agentic System/sysmlagents/`	The Python implementation: pipeline, analysis, checks, codegen, graph stores, CLI, MCP server
`ASE Agentic System/app/`	Streamlit workbench for inspecting runs
`ASE Agentic System/docs/`	Roadmap, specs RS-01 to RS-10, briefing, playbook, generated architecture page
`ASE Agentic System/workspace/`	One folder per project (gitignored: may hold client material)
`ASE School/`	Programme design, modules M01 to M12, notebooks, build scripts

The method

ASE follows the Architecture Definition Process of Rozanski and Woods (*Software Systems Architecture*, 2nd ed., ch. 7), expressed as an IDEF0 model in `methodology/ase-process-idef0.sysml`. Every ASE step realises one activity.



IDEF0 separates **inputs** (consumed; you may transform them) from **controls** (governing; not yours to change). Viewpoints, perspectives and the pattern catalogue are controls. This one distinction drives the read-only rule.

NODE	ACTIVITY	CONTROLS	OUTPUT	ASE AGENT
A1	Consolidate the inputs	Stakeholder agreement	Consolidated inputs, open areas	IntakeAgent (assists)

NODE	ACTIVITY	CONTROLS	OUTPUT	ASE AGENT
A2	Identify scenarios	Quality property requirements	Architectural scenarios	ScenarioAgent (assists)
A3	Identify relevant styles	Pattern catalogue	Candidate styles	PatternAgent (performs)
A4	Produce a candidate architecture	Styles, viewpoints, perspectives	Draft views	DesignAgent (performs)
A5	Explore the options	Viewpoints, perspectives	Refined views, decisions	AnalysisAgent (performs)
A6	Evaluate with stakeholders	Stakeholder concerns	Review comments, acceptance	EvidenceAgent (assists)
A7	Rework the architecture	Styles, viewpoints, perspectives	Reworked views	DesignAgent (performs)
A8	Revisit the requirements	Stakeholder agreement	Revised requirements	EvidenceAgent (assists); "may draft a revision, may not ratify one"

Viewpoints and perspectives

Built on ISO/IEC/IEEE 42010 constructs. A perspective earns its place only if it declares at least one computable check.

9 viewpoints (highlighted = ASE extensions):

`Context` `Functional` `Information` `Knowledge` `Concurrency` `Autonomy` `Deployment` `Operational` `Development`

13 perspectives, 34 automated checks:

PERSPECTIVE	CHECKS
Security	trust_escalation, data_exposure, tool_authorisation, sandbox_consistency, trust_declared
Performance & Scalability	latency_budget, critical_path, fanout_compounding

PERSPECTIVE	CHECKS
Availability & Resilience	articulation_points, minimum_cut, fallback_coverage, cascade_depth
Evolution	variation_rules, vocabulary_conformance, orphan_stages, dangling_exits, duplicate_capability
Regulation	requirement_traceability, verification_coverage, evidence_generable, concern_coverage, check_coverage, orphan_stakeholder, high_risk_classification
Development Resource	build_surface
Cost (extension)	cost_budget, llm_call_count, cost_sensitivity
Groundedness (extension)	provenance_paths, citation_carriers, unsourced_stages
Drift (extension)	bounded_cycles, progress_conditions, envelope_gate
Location, Usability, Accessibility, Internationalisation	Review-only, each with a recorded reason

Pattern library

`models/rag-patterns.sysml` holds six RAG patterns over one shared stage vocabulary. Patterns differ only in which stages they compose and how they wire them. Every stage carries declared estimates: `llmCalls`, `latencyMs`, `costTokens`, `fanOut`, `preservesProvenance`, `hasFallback`.

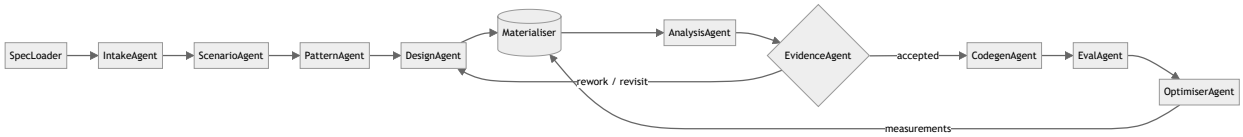
PATTERN	STAGES	LLM CALLS	LATENCY MS	TOKENS	PROVENANCE
NaiveRag	dense retriever, synthesizer	1	1,520	2,200	yes
RerankHybridRag	dense + sparse, fusion, rerank, synthesizer	1	1,715	2,200	yes
HydeRag	HyDE generator, dense, synthesizer	2	2,420	3,300	no
CorrectiveRag	dense, relevance judge, web fallback, synthesizer	2	3,020	3,000	yes

PATTERN	STAGES	LLM CALLS	LATENCY MS	TOKENS	PROVENANCE
SelfRag	dense, synthesizer, groundedness judge (loop, max 3)	6	6,660	9,300	yes
AgenticRag	orchestrator, multi-query, dense + sparse, fusion, rerank, synthesizer, judge (loop, max 5)	46	16,575	68,600	yes

Numbers are computed by the analysis from the declared stage estimates (worst case over the loop bound), not typed in. `GraphRetriever` and `QueryReformulator` are defined in the vocabulary but not yet used by any pattern.

ASE modelled in ASE

`models/ase-system.sysml` describes the target workbench, `AseWorkbench`, in the same vocabulary as client designs. Each agent appears twice: as a leaf stage with aggregate numbers, and as its own pipeline. The self-test asserts the two agree.



Run-level budgets for ASE itself: 3 hours latency (a recorded decision dated 2026-09-21, because the optimiser alone is 96% of the path) and 1,000,000 tokens. The model also records ASE's own known gaps (TrustBoundaryGap, GateVisibilityGap, HierarchyGap, ExternalKindGap) as stated positions rather than hiding them.

`models/ase-evolution.sysml` holds the workbench at each increment, I0 to I6, as its own pattern, with `@Release`, `@Delta` (stub, introduce, extend, replace, measure) and `@Realises` (which IDEF0 activity each step performs). `orchestrator.build()` generates the running LangGraph from the current increment's flows.

Knowledge graph

Two backends behind one API (`graphstore.py`): an in-memory graph for tests and offline work, and FalkorDB in Docker for the full stack. `parity()` checks they agree.

GRAPH	HOLDS	WHO WRITES
`ase` (control)	Methodology: activities, viewpoints, perspectives, concerns, checks	Nobody at run time; read-only
`ase-designs`	Every project's design, requirements, decisions, findings and runs, partitioned as `design:<project-id>`	Only `materialise.py`

The `ase` MCP server exposes typed tools (`ase_step` , `ase_controls` , `ase_viewpoint` , `ase_perspective` , `ase_gap` , …) rather than raw Cypher, by design, so agents can read the method but never write to it.

Evolution, I0 to I6

Seven increments. Each ends with a working product that does strictly more than the last, is described by its own SysML model, and passes every spec assigned to it or to any earlier increment.

INC	GOAL	SPECS	WHAT CHANGES
I0 ✓	Walking skeleton: one command runs the pipeline on RS-01; the self-model exists	RS-01	Scripted answers, keyword selector, pattern copy, sweep only, stub evaluator. Delivered 2026-09-21
I1	Intake: find what is missing against the methodology, ask, record assumptions	RS-01 full, RS-02	IntakeAgent; deterministic gap detection; the LLM only words the questions
I2	Pattern selection from the graph; design instantiation	RS-02, RS-03	ScenarioAgent, PatternAgent, DesignAgent; `@PatternForm` in the library
I3	Analysis, options, negotiation, conditional edges	RS-04 to RS-06	AnalysisAgent, EvidenceAgent; first rework loop (max 3); options table
I4	Spec in, deployed and measured agent out	RS-01 to RS-05 at L3	Real retrieval runtime, eval datasets, GEPA with held-out gate, Deployer, measured numbers written back

INC	GOAL	SPECS	WHAT CHANGES
I5	Vocabulary extension and pattern composition	RS-07, RS-08	Composer; trust and tenancy; runtime patterns in deployment templates; promotion workflow
I6	Multi-agent composition and governed self-improvement	RS-09, RS-10	Agent-as-part vocabulary, evidence packs, release gate, canary deploys; ASE optimises its own prompts

Specs move through four tiers: **1** runs end to end, **2** finds the gap and gets a decision, **3** negotiates or extends the vocabulary, **4** multi-agent, regulated evidence, the method applied to itself.

Reference specs

Each spec has a submitted part (given to ASE, "deliberately incomplete, sometimes inconsistent, occasionally infeasible") and a hidden test oracle with scripted stakeholder answers. They are the acceptance tests of the factory.

SPEC	TIER	EXPECTED DESIGN	WHAT IT PROVES
RS-01 HR Policy Assistant	1	NaiveRag	The full happy path
RS-02 Equipment Manual Assistant	1	RerankHybridRag	Reasoned selection, parallel branches, a version filter the library lacks
RS-03 Research Abstracts Explorer	2	HydeRag, gap signed off	Surfacing an unallocated requirement and getting a decision
RS-04 Support KB with Web Fallback	2	CorrectiveRag	Conditional branch, fallback, first security finding
RS-05 Clinical Guideline Q&A	2	SelfRag	Bounded loop, abstention, measured feedback (flagship)
RS-06 Enterprise Knowledge Copilot	3	Infeasible as specified	Proving infeasibility and negotiating instead of designing

SPEC	TIER	EXPECTED DESIGN	WHAT IT PROVES
RS-07 Multi-tenant Contract Assistant	3	RerankHybridRag + GraphRetriever + tenancy	Security end to end; vocabulary extended with a reason
RS-08 Conversational Onboarding	3	Composed, two workload classes	New stage kinds, response-time classes
RS-09 Competitive Intelligence Agent	4	Multi-agent orchestrator–worker	Concurrency and autonomy views, network-science checks
RS-10 Regulated Financial Advice	4	SelfRag + human oversight + evidence pack	Full regulation perspective; gated optimisation

Pattern catalogue

Beyond RAG, the library will grow from a catalogue of 55 agentic patterns (Arsanjani and Bustos, *Agentic Architectural Patterns for Building Multi-Agent Systems*), held in `AI Skills/skills/ase-arch-patterns/references/patterns.json`. They are ordered single-agent first: a pattern is built only after the patterns it requires.

STAGE	THEME	PATTERNS	EXAMPLES
P1	Minimal single agent	6	Single Agent Baseline, Sensing with RAG, Watchdog Timeout, Agent Calls Human
P2	Reasoning and self-correction	6	Structured Reasoning and Self-Correction, Instruction Fidelity Auditing
P3	Auditable and guarded	6	Agent Self-Defense, Authentication and Authorization, Sandboxing, Rate Limiting
P4	Evaluated and optimised	4	Custom Evaluation Metrics, Red Teaming, Tokenomics
P5	Production single agent	6	Fallback Model, Incremental Checkpointing, Event-Driven Reactivity
P6	Top-down multi-agent	9	Agent Router, Supervisor, Multi-Agent Planning, Tool and Agent Registry
P7	Meta-agents, resilience, governance of teams	10	Supervision Tree, Blackboard, Canary Agent Testing

STAGE	THEME	PATTERNS	EXAMPLES
R8	Decision self-coordinatory	8 Consensus Net	`scripts/catalogue.py`
	self-improvement	Measuring Business Value	

The catalogue is itself a control: it never edit it during a project. `coverage` reports which stage a system has reached and which prerequisites are missing.

How we work

Item by item, with a review gate

1. Every item has a written goal, the files it touches, a definition of done, and its model delta if it changes ASE's agents, flows or tools.
2. The model delta (`ase-system.sysml` , `ase-evolution.sysml`) lands before the code.
3. One item per pull request, rebased on current main, small enough to review in one sitting.
4. Verify (`cli selftest` plus tests), report, and wait for approval before the next item.

Three test layers

LAYER	WHAT IT CHECKS	WHERE
L1 Gate and self-model	Models pass the structural gate; the sweep has no errors; the orchestrator regenerates identically; every step realises an activity	`cli selftest` , `tests/test_selfmodel.py`
L2 Spec oracles	Each RS-xx end to end with the scripted stakeholder. What the current increment cannot meet is `xfail` naming the increment that will; burn them down, never delete assertions	`tests/specs/test_rs_xx.py`
L3 Generated product	The generated agent on real data clears its metric thresholds; from I4 through its deployed endpoint	marked `slow`

Tests run with no FalkorDB and no model calls. If your change needs a key or a database to pass its tests, it's not done.

Pull request checklist

- Rebased on current main, no conflicts.

- `models/` , `methodology/` and skills unchanged, or the change is a reviewed library change and says so.
- No provider SDK imported in pipeline, agent or codegen code.
- A new step has an `@Realises` entry, and `cli selftest` is OK.
- Offline tests pass with no key and no database.
- Generated files regenerated, not edited.
- Runs go through projects (`workspace/<id>/runs/`), not ad-hoc folders.

Your first week

Python environment: conda `DATAENLIGHT_AI_ARCH_PATTERNS` , with the root `requirements.txt` plus `ASE Agentic System/requirements.txt` . On Windows set `PYTHONIOENCODING=utf-8` . From `ASE Agentic System/` :

```
python -m sysmlagents.cli selftest                    # L1: should end with OK
python -m pytest tests -m "not slow" -c tests/pytest.ini
python -m sysmlagents.cli run docs/specs/RS-01-hr-policy-assistant.md --offline
python -m sysmlagents.cli trace RS-01                # the steps of that run
python -m sysmlagents.cli explain RS-01
python -m sysmlagents.cli up                         # Docker + FalkorDB + graphs
```

1. **Day 1.** Read `CLAUDE.md` , then this handbook, then `docs/ROADMAP.md` sections 2 and 3.
2. **Day 2.** Run the commands above. Open the workbench and follow one RS-01 run step by step in the Step Inspector.
3. **Day 3.** Read `models/rag-patterns.sysml` and the `AseWorkbench` part of `models/ase-system.sysml` . Compare one pattern's computed numbers with the table above.
4. **Day 4.** Read RS-01 and RS-02 in `docs/specs/` , including the hidden oracle, and `tests/specs/test_rs_01.py` .
5. **Day 5.** Pick a first item with your reviewer and write its one-paragraph item spec before any code.

> **Known documentation drift.** A few documents disagree with the code; trust the code and models. `ase-system.sysml` , `README.md` and the roadmap say 36 checks; there are 34. The IDEF0 file calls two agents ConsolidationAgent and ViewAuthoringAgent; the system model calls them IntakeAgent and DesignAgent. The Deployer is in the roadmap but not yet in the models. `docs/ase-architecture.html` was last generated on 21 September 2026.

Glossary

Control	Something an activity must obey and may not change: viewpoints, perspectives, pattern library, catalogue, stakeholder agreement.
Input	Something an activity consumes and may transform: the spec, scenarios, draft views.
Spec (RS-xx)	A reference specification with a submitted part and a hidden test oracle.
Project	One design in the workspace: `spec/`, `inputs/`, `design/`, `runs/`, `journal.jsonl`. Keys its graph partition.
Stage	One step of an agent's pipeline, with declared estimates (calls, latency, tokens, fan-out, provenance, fallback).
Pattern	A named composition of stages and flows, with requirements it satisfies.
@Extension	A recorded departure from the library, with a reason, on a design. Promoted into the library only by a human commit.
@Realises	Metadata linking an ASE step to the IDEF0 activity it performs.
Increment (I0 to I6)	One release of the workbench, modelled in `ase-evolution.sysml`.
Gate	The structural SysML check (`check_model.py`) that decides well-formedness before anything reads a model.
Sweep	Running all 34 perspective checks over a model.
Oracle	The expected outcome for a spec: pattern, questions asked and not asked, findings, decisions, metrics.
GEPA	Reflective prompt optimisation for DSPy programs; used with a held-out acceptance gate.